LAB ASSIGNMENT -4
(ADITYA ONAM, 2301MC60)

Project Report: AI Scheduling & Optimization Algorithms

Executive Summary

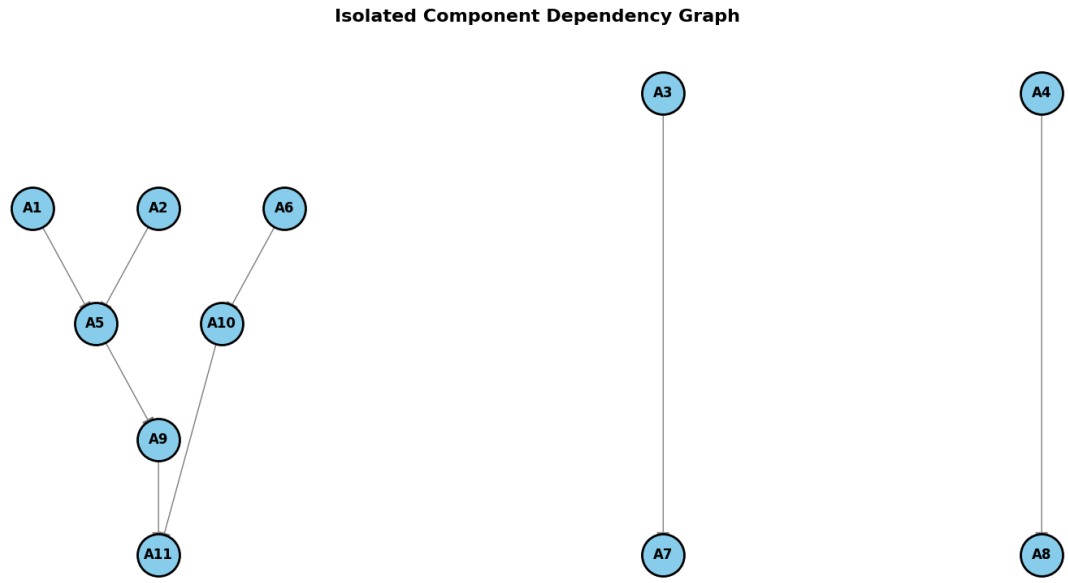
This report details the implementation and comparative analysis of two Artificial Intelligence search strategies—Greedy Search and A* (A-Star) Search—applied to a complex task scheduling problem. The objective is to assign dependent tasks to specific days while respecting prerequisite constraints, group-size limits, and optimizing for the lowest total catering cost using a "Global Fixed Menu" rule.

Part 1: Problem Instances & Topological Testing

To ensure the scheduling algorithms were robust and not over-optimized for a single scenario, they were tested against three distinct topological structures. Each instance tests a different edge case in graph traversal and cost optimization.

1.1 Instance 1: The Original Sample (Hybrid Tree)

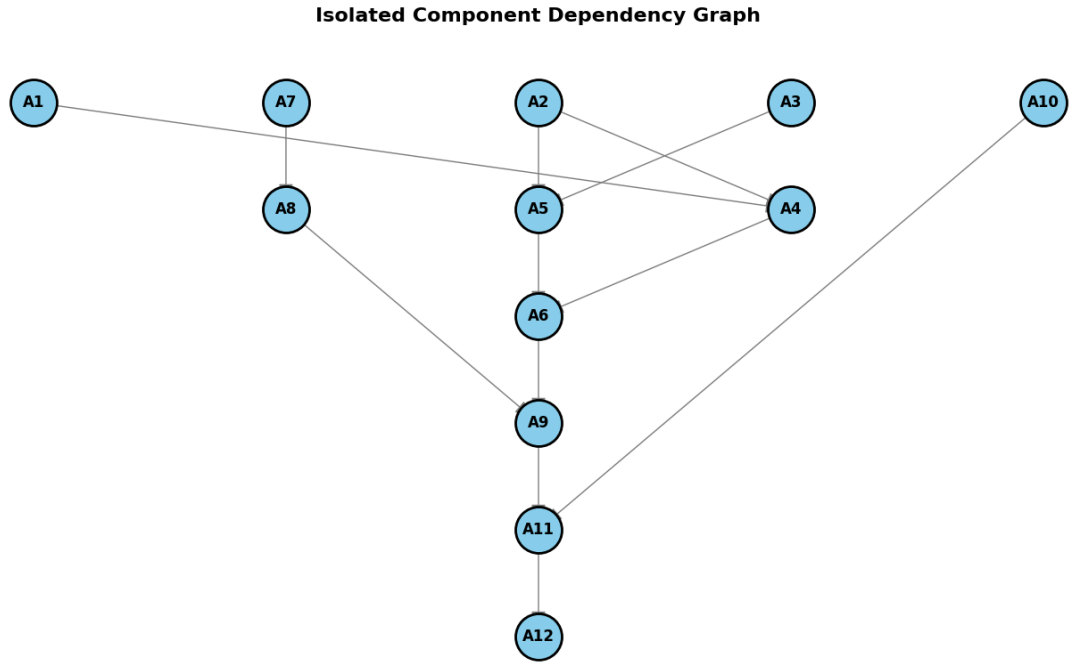
- **Structure:** A balanced, hybrid tree with multiple starting points, parallel branches, and a strict bottleneck at the end (Task A11).
- **Parameters:** Group Size (G) = 2. Equal food costs ($1 each).
- **Testing Goal:** Evaluates the algorithm's general baseline ability to juggle parallel task groupings while adhering strictly to the maximum group size limit.



-

1.2 Instance 2: The Linear Pipeline (Depth Test)

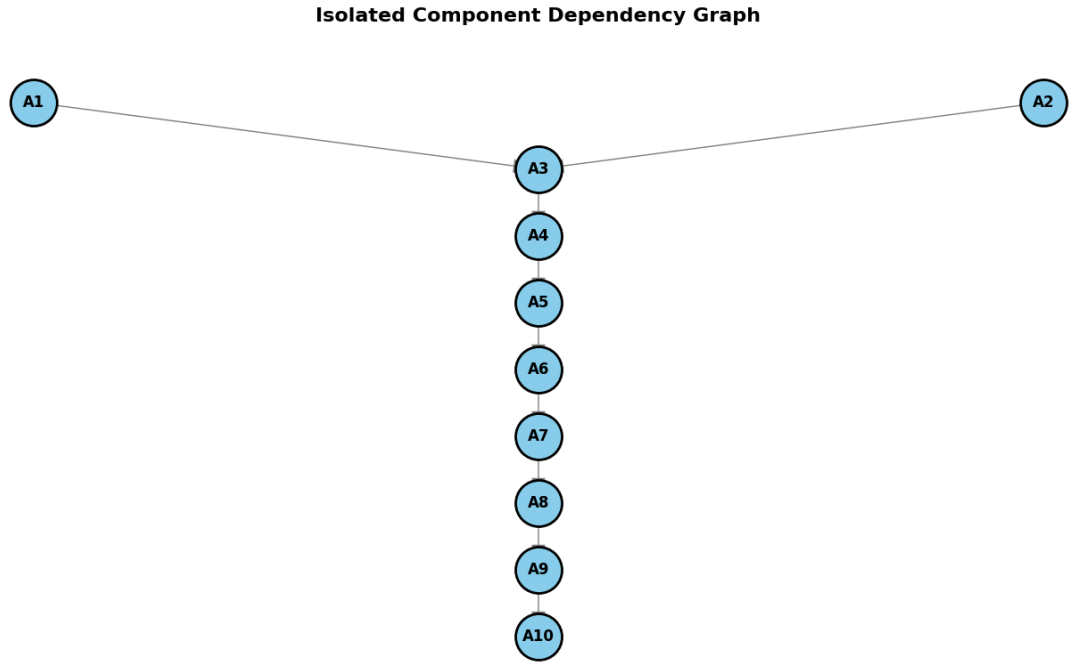
- **Structure:** A straight, sequential line ($A1 \rightarrow A2 \rightarrow A3 \rightarrow A4 \rightarrow A5$).
- **Parameters:** Group Size (G) = 2. Variable food costs.
- **Testing Goal:** This acts as a strict "Depth Test." Even though the workforce can handle 2 tasks per day, the strict sequential dependencies mean only 1 task can ever be scheduled per day. It verifies that the A* heuristic correctly calculates depth constraints over volume capacity.



-

1.3 Instance 3: The Wide Tree (Parallel Cost Test)

- **Structure:** A highly parallel graph where multiple tasks unlock simultaneously.
- **Parameters:** Group Size (G) = 3. Extreme food cost variance (Steak = $10, Bread = $1).
- **Testing Goal:** This tests the algorithm's ability to prioritize the "Global Max Menu" rule over sheer speed. A basic algorithm will try to schedule three tasks immediately to finish fast, inadvertently grouping Steak and Bread on Day 1 and causing a massive permanent daily cost. An optimal algorithm will realize it is cheaper to group identical foods together and extend the project timeline.



-

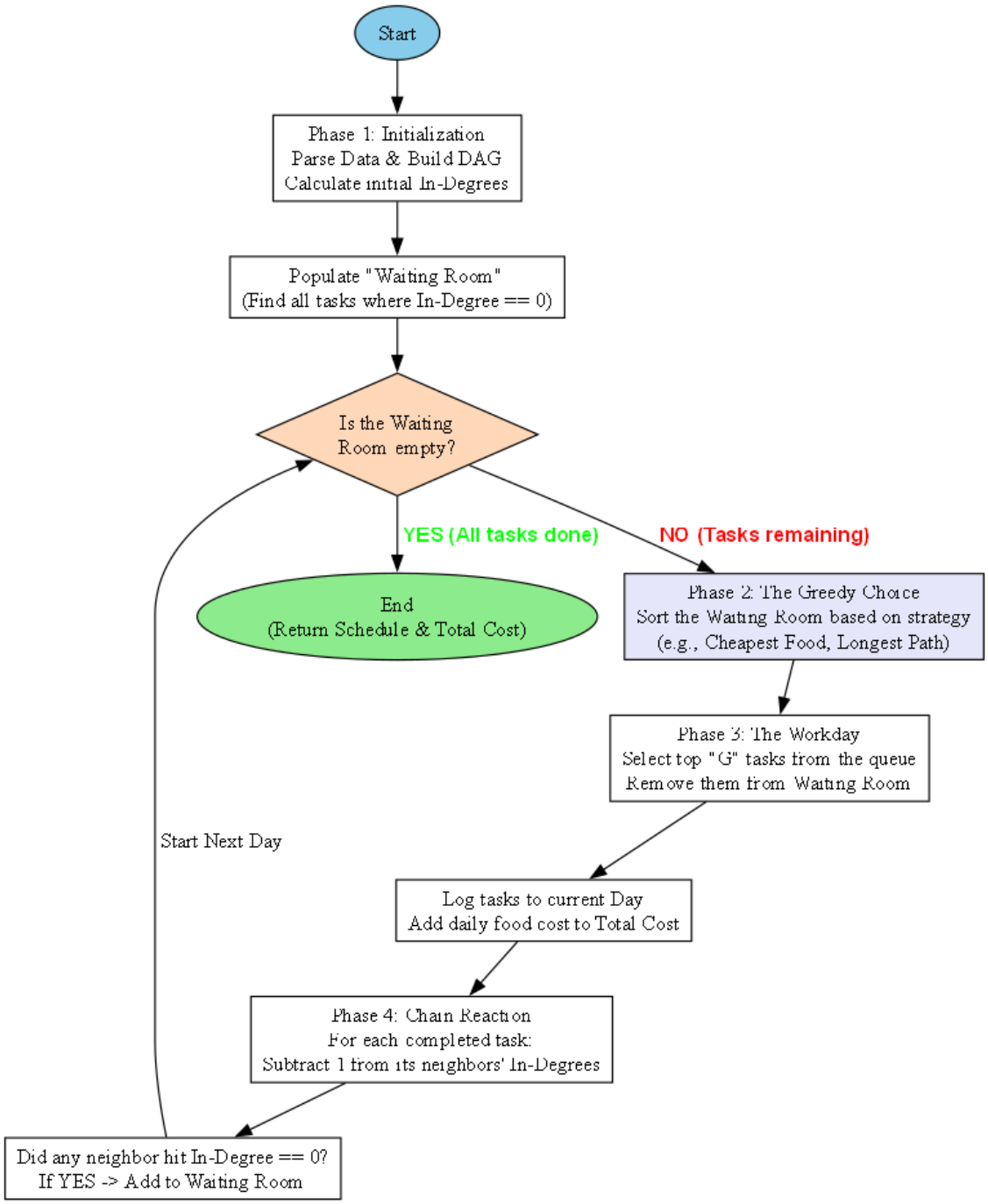
Part 2: Task 1 - Greedy Scheduling Algorithm

2.1 Overview of the Greedy Approach

The Greedy algorithm attempts to build a valid schedule by making the locally optimal choice at each step (each day). It traverses the Directed Acyclic Graph (DAG) of task dependencies, identifies all currently available tasks (in-degree of 0), and selects a subset of tasks up to the maximum Group Size limit based on a specific sorting heuristic.

2.2 Implementation Flow

The algorithm operates on a day-to-day loop, parsing tasks from an initial "Waiting Room" to a completed state, reducing the dependency requirements of subsequent tasks in a chain-reaction format.



2.3 Greedy Execution Results (Instance 1)

For the baseline test case (11 Tasks, Group Size: 2), four distinct greedy sorting strategies were applied. The isolated daily food cost across all strategies was identical:

- **Total Days Required:** 6 Days
- **Total Isolated Food Cost:** $11
- **Observation:** Because the Greedy algorithm only looks at the immediate day, all heuristic strategies (Cheapest Food First, Deepest Critical Path, Most Immediate Successors, Highest Food Frequency) resulted in the exact same day-count and localized cost footprint. It failed to account for the Global Menu penalty.

---

*Part 3: Task 2 - A Search Optimization**

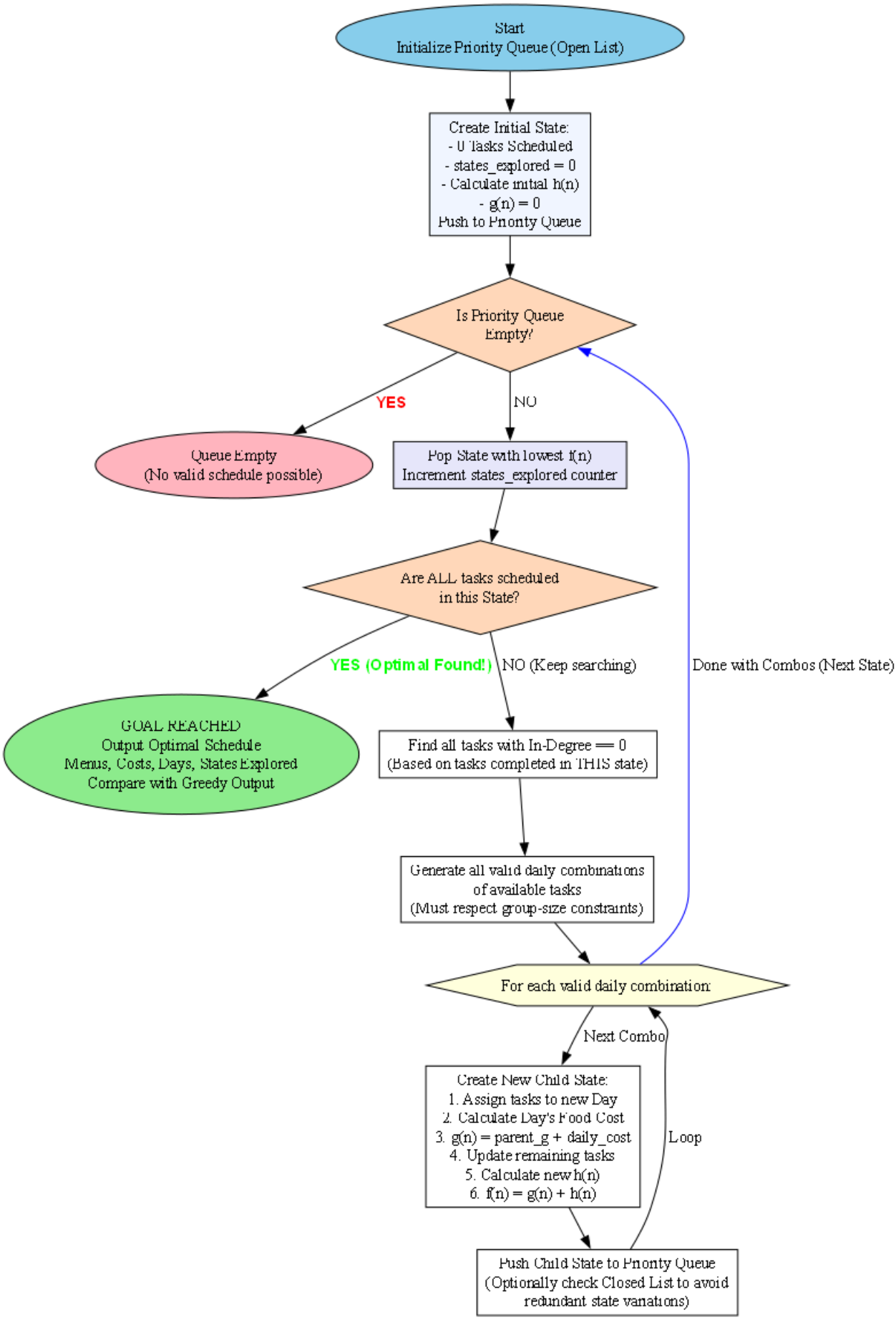
*3.1 Overview of the A Approach**

Unlike the Greedy algorithm which finds *a* valid path, the A* search algorithm is designed to find the *absolute optimal* path. In this context, it searches through a "state-space tree" of possible schedules to minimize a strict global constraint: **The Global Max Menu Cost**. Under this rule, the maximum quantity of any food item required on a single day dictates the fixed daily cost for the *entire* duration of the project.

A* evaluates states using the formula $f(n) = g(n) + h(n)$:

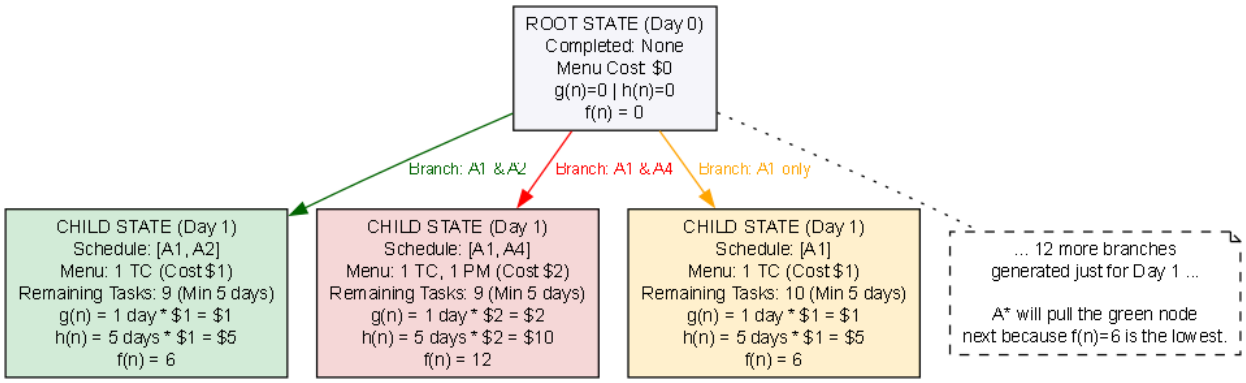
- **$g(n)$ (Path Cost):** The current day multiplied by the current global max menu cost.
- **$h(n)$ (Heuristic):** An admissible estimate of remaining costs based on the maximum of volume bounds (remaining tasks / group size) and depth bounds (longest remaining prerequisite chain).

*3.2 A Implementation Flow**


Start
Initialize Priority Queue (Open List)
Create Initial State:
- 0 Tasks Scheduled
- states_explored = 0
- Calculate initial h(n)
- g(n) = 0
Push to Priority Queue
Is Priority Queue Empty?
YES
NO
Queue Empty
(No valid schedule possible)
Pop State with lowest f(n)
Increment states_explored counter
Are ALL tasks scheduled in this State?
YES (Optimal Found!)
NO (Keep searching)
Done with Combos (Next State)
GOAL REACHED
Output Optimal Schedule
Menus, Costs, Days, States Explored
Compare with Greedy Output
Find all tasks with In-Degree == 0
(Based on tasks completed in THIS state)
Generate all valid daily combinations of available tasks
(Must respect group-size constraints)
For each valid daily combination:
Next Combo
Create New Child State:
1. Assign tasks to new Day
2. Calculate Day's Food Cost
3. g(n) = parent_g + daily_cost
4. Update remaining tasks
5. Calculate new h(n)
6. f(n) = g(n) + h(n)
Loop
Push Child State to Priority Queue
(Optionally check Closed List to avoid redundant state variations)

3.3 State-Space Expansion

To find the optimal schedule, A* must generate and evaluate different combinations of daily task assignments. Branching occurs when multiple tasks are available, forcing the algorithm to weigh the cost of grouping certain tasks together versus spreading them out.



(Paste the Partial Search Tree / Level 1 graph showing the Root Node splitting into "Good" and "Bad" schedule branches here)

Part 4: Statistical Comparison & Analysis

4.1 Experimental Results (Instance 1)

The following data compares the best Greedy schedule against the optimal A* schedule, evaluated under the strict Global Max Menu catering rules.

- *A States Explored:** 315 states evaluated before finding the mathematical optimum.
- *A Global Menu Required:** <1 TC, 1 DF, 1 PM, 1 GJ> (Base cost: $4/day)

Metric	**Greedy Algorithm Baseline**	**A* Search (Optimal)**	**Difference**
Total Days Taken	6 Days	7 Days	**+1 Day**
Global Menu Cost/Day	$5 per day	$4 per day	**-$1/day**

Total Project Cost	$30	$28	**-$2 Total**

*4.2 A Optimal Schedule Breakdown (Instance 1)**

- **Day 1:** A6 (Cost: 1)
- **Day 2:** A1 (Cost: 1)
- **Day 3:** A3 (Cost: 1)
- **Day 4:** A7, A2 (Cost: 2)
- **Day 5:** A5, A4 (Cost: 2)
- **Day 6:** A9, A10 (Cost: 2)
- **Day 7:** A8, A11 (Cost: 2)

4.3 Conclusion & Algorithm Trade-offs

The comparison reveals a classic optimization trade-off that the Greedy algorithm fundamentally failed to detect.

Because the Greedy algorithm "greedily" packed tasks together to finish the project as fast as possible (6 days), it accidentally created a "spike" in daily food requirements on its combined days, pushing the Global Fixed Menu cost to $5/day. Total cost: $30.

The *A Algorithm**, guided by its heuristic and global cost evaluation, recognized that packing tasks too densely was a financial trap. It strategically chose to *slow down* the schedule, doing only one task on Days 1, 2, and 3. By extending the project length to 7 days, it successfully suppressed the maximum daily food requirement, dropping the Global Menu cost to $4/day. Total cost: $28.

Final Verdict: While the Greedy algorithm is computationally cheaper and finishes the project faster, A* successfully navigated the complex state-space to find the true mathematical optimum, achieving a lower total financial cost by intelligently balancing project duration against the global menu constraint.